

UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

Facultad de Ingeniería
Escuela de Ciencias y Sistemas



Manual Técnico

GoLite - Proyecto Fase 1

Nombre Juan Francisco Ramirez de Paz

Fecha 12 de junio de 2026

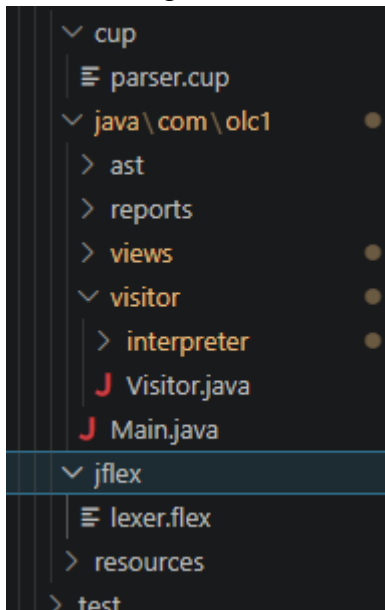
Introducción

El presente manual técnico describe la estructura interna del proyecto GoLite, así como las herramientas utilizadas para su desarrollo y el funcionamiento general del analizador léxico, sintáctico, semántico e intérprete.

El proyecto fue desarrollado en Java, utilizando JFlex para el análisis léxico, CUP para el análisis sintáctico y Java Swing para la interfaz gráfica.

Tecnologías utilizadas

- Java Maven
- JFlex
- CUP
- Java Swing



Estructura del proyecto

La estructura principal del proyecto es la siguiente:

```
src/main/java |— analisis |— com/olc1/ast |— com/olc1/reports |—  
com/olc1/views |— com/olc1/visitor
```

```
src/main/jflex |— lexer.flex
```

```
src/main/cup |— parser.cup 4. Análisis léxico
```

El análisis léxico se realiza con JFlex mediante el archivo:

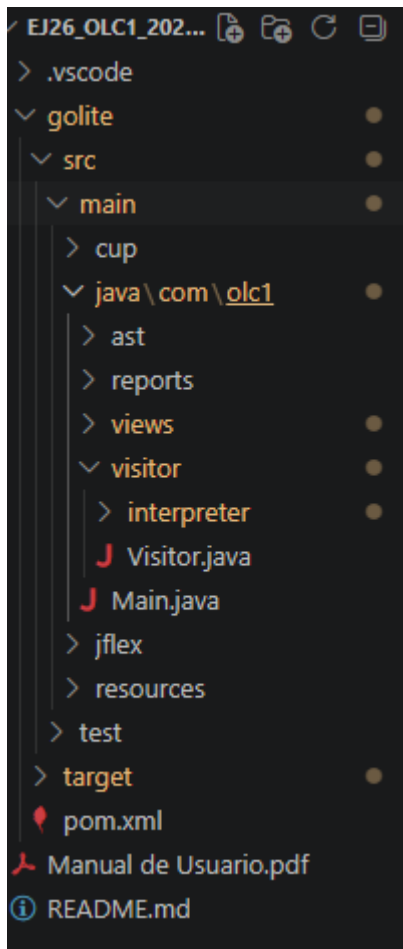
```
src/main/jflex/lexer.flex
```

Este archivo reconoce los lexemas del lenguaje GoLite y los convierte en tokens que luego son enviados al parser.

Entre los tokens reconocidos se encuentran:

Identificadores Enteros Decimales Cadenas Rune Palabras reservadas Operadores aritméticos Operadores lógicos Operadores relacionales Símbolos de agrupación Funciones embebidas

Además, el lexer registra errores léxicos y almacena la tabla de tokens para el reporte correspondiente.



Análisis sintáctico

El análisis sintáctico se realiza con CUP mediante el archivo:

src/main/cup/parser.cup

El parser recibe los tokens generados por el lexer y valida que cumplan con la gramática del lenguaje GoLite.

Cuando la entrada es correcta, se construye un Árbol de Sintaxis Abstracta o AST.

El parser reconoce instrucciones como:

Declaración de variables Asignaciones Impresiones If / Else if / Else For Break
Continue Bloques de sentencias 6. Árbol de Sintaxis Abstracta

El AST está formado por clases Java ubicadas en:

src/main/java/com/olc1/ast

Estas clases representan las instrucciones y expresiones del lenguaje.

Ejemplos de nodos del AST:

Add Sub Mul Div Mod VarDecl Assign AllLocate IfNode WhileFor Block Imprimir EmbeddedFunction

Cada nodo implementa la interfaz base del AST y utiliza el patrón Visitor para ser interpretado.

```
FOR_INIT ::= id:n assign EXPRESION:e          {: RESULT = new Assign(n, e, nleft, nright); :}  
| id:i allocate EXPRESION:e                  {: RESULT = new Alllocate(i, e, ileft, iright); :}  
;  
  
FOR_UPDATE ::= id:i allocate EXPRESION:e      {: RESULT = new Alllocate(i, e, ileft, iright); :}  
| id:i plusAssign EXPRESION:e                {: RESULT = new Alllocate(i, new Add(new VarRef(i, ileft), new VarRef(i, iright)); :}  
| id:i minusAssign EXPRESION:e               {: RESULT = new Alllocate(i, new Sub(new VarRef(i, ileft), new VarRef(i, iright)); :}  
| id:i increment                             {: RESULT = new Alllocate(i, new Add(new VarRef(i, ileft), new Literal(1)); :}  
| id:i decrement                             {: RESULT = new Alllocate(i, new Sub(new VarRef(i, ileft), new Literal(1)); :}  
;  
  
INSTRUCCION ::= imprimir lparen rparen scol    {: RESULT = new Imprimir(new java.util.ArrayList<ASTNODE>()); :}  
| fmtPrintln lparen rparen scol               {: RESULT = new Imprimir(new java.util.ArrayList<ASTNODE>()); :}  
| imprimir lparen EXPRESIONES:e rparen scol   {: RESULT = new Imprimir(e); :}  
| fmtPrintln lparen EXPRESIONES:e rparen scol {: RESULT = new Imprimir(e); :}  
| kwVar id:n type:t allocate EXPRESION:e scol {: RESULT = new VarDecl(n, t, e, nleft, nright); :}  
| kwVar id:n type:t scol                      {: RESULT = new VarDecl(n, t, null, nleft, nright); :}  
| id:n assign EXPRESION:e scol                {: RESULT = new Assign(n, e, nleft, nright); :}  
| id:i allocate EXPRESION:e scol              {: RESULT = new Alllocate(i, e, ileft, iright); :}  
;  
  
ELIF ::= ELIF:el kwElse kwIf EXPRESION:e lbrace INSTRUCCIONES:i rbrace {: el.add(new ElifNode(e, i)); RESULT = el; :}  
| kwElse kwIf EXPRESION:e lbrace INSTRUCCIONES:i rbrace               {: RESULT = new ElifNodes(new ElifNode(e, i)); :}  
| ELIF:el kwElse:els lbrace INSTRUCCIONES:i rbrace                    {: el.add(new ElifNode(new BoolLiteral(true, elsleft, elsrigh), i)); :}  
| kwElse:els lbrace INSTRUCCIONES:i rbrace                            {: RESULT = new ElifNodes(new ElifNode(new BoolLiteral(true, elsleft, elsrigh), i)); :}  
;  
  
EXPRESIONES ::= EXPRESIONES:l comma EXPRESION:e  {: l.add(e); RESULT = l; :}  
| EXPRESION:e {: java.util.List<ASTNODE> list = new java.util.ArrayList<>(); list.add(e); RESULT = list; :}  
;  
  
EXPRESION ::= minus EXPRESION:a                  {: RESULT = new Negate(a); :}%prec UMENOS  
| EXPRESION:a plus EXPRESION:b                  {: RESULT = new Add(a, b); :}  
| EXPRESION:a minus EXPRESION:b                 {: RESULT = new Sub(a, b); :}  
| EXPRESION:a times EXPRESION:b                 {: RESULT = new Mul(a, b); :}  
| EXPRESION:a slash EXPRESION:b                 {: RESULT = new Div(a, b); :}  
| EXPRESION:a mod EXPRESION:b                   {: RESULT = new Mod(a, b); :}%prec mod  
| EXPRESION:a equal EXPRESION:b                 {: RESULT = new Compare(a, b, "=="); :}  
| EXPRESION:a notEqual EXPRESION:b              {: RESULT = new Compare(a, b, "!="); :}  
| EXPRESION:a lowertag EXPRESION:b              {: RESULT = new LowerTag(a, b); :}  
| EXPRESION:a greatertag EXPRESION:b            {: RESULT = new Compare(a, b, ">"); :}  
| EXPRESION:a lowerEqual EXPRESION:b            {: RESULT = new Compare(a, b, "<="); :}  
| EXPRESION:a greaterEqual EXPRESION:b          {: RESULT = new Compare(a, b, ">="); :}  
| EXPRESION:a and EXPRESION:b                  {: RESULT = new Logical(a, b, "&&"); :}  
| EXPRESION:a or EXPRESION:b                   {: RESULT = new Logical(a, b, "||"); :}  
| not EXPRESION:e                              {: RESULT = new Logical(e, null, "!"); :}%prec not  
| EXPRESION:e                                  {: RESULT = new Literal(e); :}
```

Intérprete

La interpretación del AST se realiza mediante la clase:

src/main/java/com/olc1/visitor/interpreter/InterpreterVisitor.java

Esta clase recorre los nodos del AST y ejecuta las instrucciones del programa.

Entre sus responsabilidades están:

Evaluar expresiones Ejecutar instrucciones Controlar ámbitos Validar tipos Manejar errores semánticos Controlar sentencias break y continue Ejecutar funciones embebidas 8. Manejo de ámbitos

El manejo de ámbitos se realiza mediante la clase Enviroment.

Cada bloque de sentencias puede crear un nuevo entorno. Esto permite manejar correctamente variables locales, shadowing y visibilidad de variables dentro y fuera de bloques.

```
package com.olc1.visitor.interpreter;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

import com.olc1.ast.ASTNODE;
import com.olc1.ast.exp.*;
import com.olc1.ast.stm.*;
import com.olc1.reports.GoliteError;
import com.olc1.visitor.Visitor;
import com.olc1.visitor.interpreter.value.*;
import com.olc1.visitor.interpreter.transfer.BreakException;
import com.olc1.visitor.interpreter.transfer.ContinueException;

public class InterpreterVisitor implements Visitor<ValueWrapper> {
    public String output = "";
    private final ValueWrapper defaultVoid = new VoidValue(-1, -1);
    private Enviroment enviroment = new Enviroment();
    private int loopDepth = 0;
    public final List<GoliteError> errors = new ArrayList<>();

    private void executeInNewScope(ASTNODE body) {
        Enviroment previous = this.enviroment;

        try {
            this.enviroment = new Enviroment(previous);
            Visit(body);
        } finally {
            this.enviroment = previous;
        }
    }
}
```

Tipos de datos

El lenguaje implementa tipos primitivos mediante clases de valor.

Entre los tipos soportados se encuentran:

int float64 string bool rune nil

Cada tipo se representa mediante una clase que implementa la interfaz ValueWrapper.

```
package com.olc1.visitor.interpreter.value;

public sealed interface ValueWrapper
    permits IntValue, DecimalValue, VoidValue, BoolValue, StringValue, NilValue, RuneValue {

    int line();
    int column();
    String getTypeName();
}
```

Manejo de errores

El proyecto maneja tres tipos de errores:

Errores léxicos Errores sintácticos Errores semánticos

Los errores se almacenan en objetos de la clase:

com.olc1.reports.GoliteError

Cada error contiene:

Tipo Descripción Línea Columna

Estos errores se muestran en una ventana de reporte dentro de la interfaz gráfica.

```
package com.olc1.reports;

✓ public class GoliteError {
    private final String type;
    private final String description;
    private final int line;
    private final int column;

    ✓ public GoliteError(String type, String description, int line, int column) {
        this.type = type;
        this.description = description;
        this.line = line;
        this.column = column;
    }

    ✓ public String getType() {
        return type;
    }

    ✓ public String getDescription() {
        return description;
    }

    ✓ public int getLine() {
        return line;
    }

    ✓ public int getColumn() {
        return column;
    }
}
```

Reporte de tokens

El reporte de tokens se maneja mediante la clase:

com.olc1.reports.TokenReport

Cada token registrado contiene:

Lexema Tipo de token Línea Columna

La información se muestra en una tabla dentro de una ventana de reportes.


```

package com.olc1.reports;

public class TokenReport {
    private final String lexeme;
    private final String tokenType;
    private final int line;
    private final int column;

    public TokenReport(String lexeme, String tokenType, int line, int column) {
        this.lexeme = lexeme;
        this.tokenType = tokenType;
        this.line = line;
        this.column = column;
    }

    public String getLexeme() {
        return lexeme;
    }

    public String getTokenType() {
        return tokenType;
    }

    public int getLine() {
        return line;
    }

    public int getColumn() {
        return column;
    }
}

```

Gramática BNF resumida

::=

::= |

::= "fmt.Println" "(" ")" | "print" "(" ")" | "var" id "=" | "var" id | id "!=" | id "=" | id "+=" | id
 "-=" | "if" "{" "}" | "if" "{" "}" | "for" "{" "}" | "for" ";" ";" "{" "}" | "break" | "continue" | "{" "}"

::= "else" "if" "{" "}" | "else" "{" "}" | ϵ

::= "int" | "float64" | "string" | "bool" | "rune"

::= id "!=" | id "="

::= id "=" | id "+=" | id "-=" | id "++" | id "--"

::= "," |

```
::= "+" | "-" | "*" | "/" | "%" | "==" | "!=" | "<" | ">" | "<=" | ">=" | "&&" | "||" | "!" | "-" | "(" ")"  
| id | entero | decimal | cadena | rune | "true" | "false" | "nil" | "strconv.Atoi" "(" ")" |  
"strconv.ParseFloat" "(" ")" | "reflect.TypeOf" "(" ")" ".string" "(" ")"
```

::= ";" | salto_linea 13. Compilación del proyecto

Para compilar el proyecto se utiliza Maven:

```
mvn clean compile
```

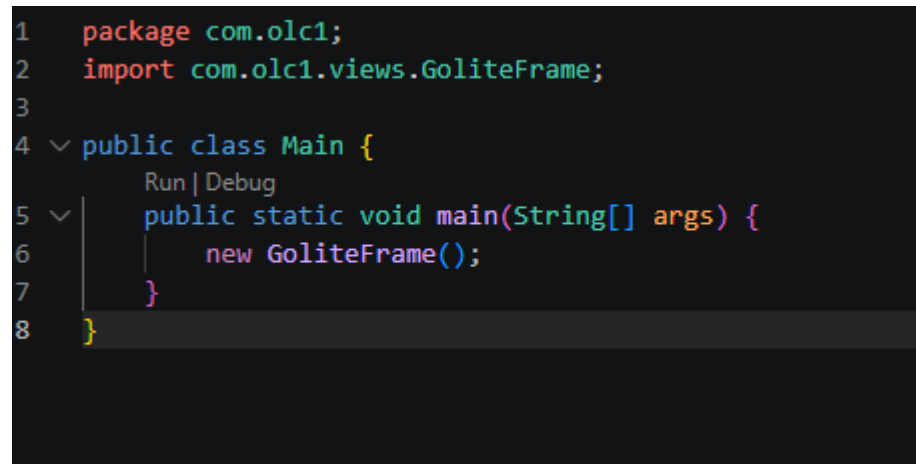
Este comando genera los analizadores y compila el proyecto completo.

Ejecución del proyecto

La ejecución se realiza desde la clase principal:

```
src/main/java/com/olc1/Main.java
```

Al ejecutar esta clase se inicia la interfaz gráfica del intérprete.



```
1 package com.olc1;  
2 import com.olc1.views.GoliteFrame;  
3  
4 public class Main {  
5     Run | Debug  
6     public static void main(String[] args) {  
7         new GoliteFrame();  
8     }  
}
```

Conclusión

El proyecto GoLite implementa las fases principales de un intérprete básico: análisis léxico, análisis sintáctico, análisis semántico y ejecución del AST. Además, incluye una interfaz gráfica y reportes que permiten visualizar errores y tokens reconocidos durante el análisis.